-=LAPORAN PRAKTIKUM

POSTTEST 7

ALGORITMA PEMROGRAMAN DASAR



Disusun oleh:

Nama (2609106097)

Kelas (C2 ‘26)

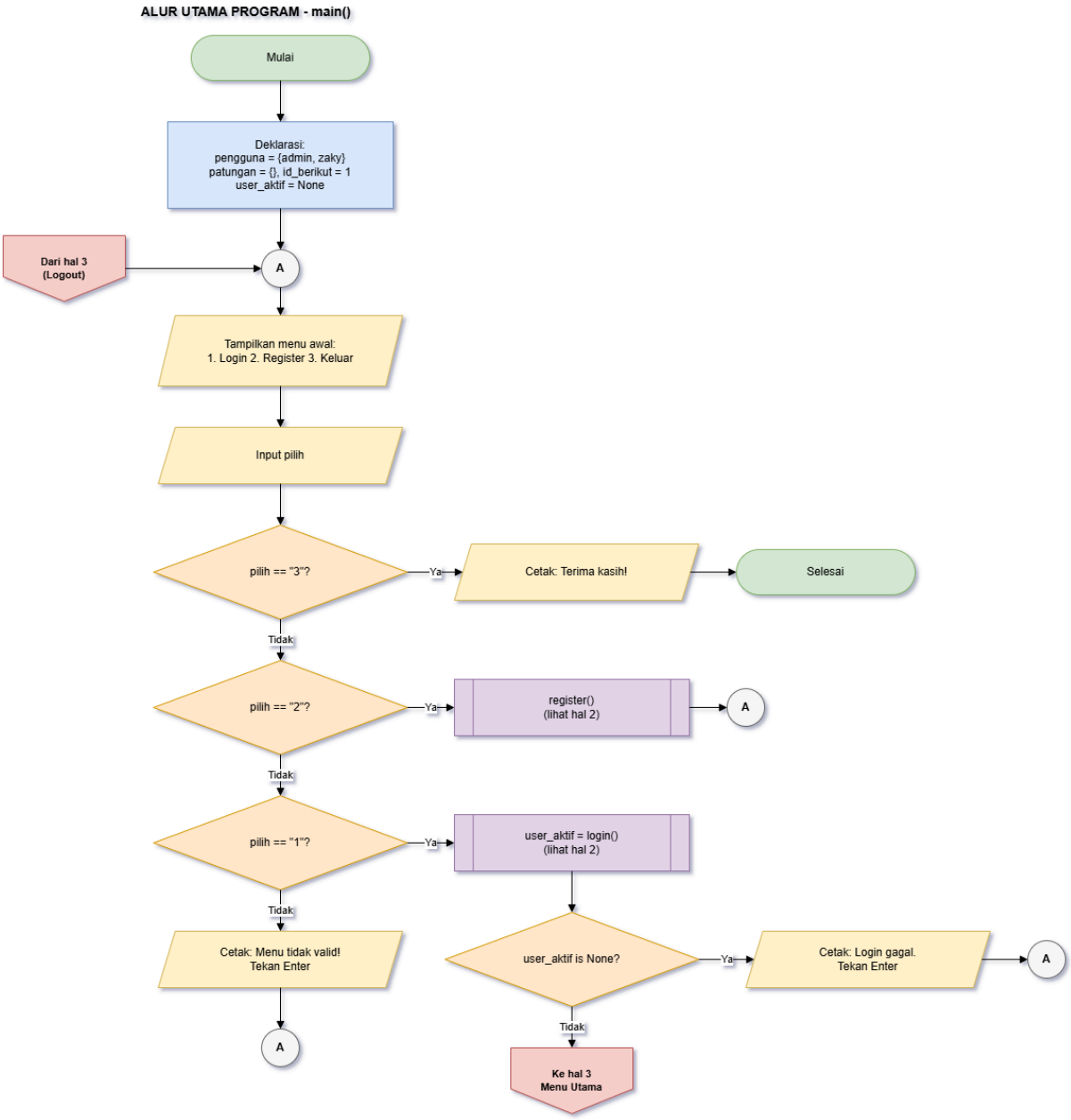
PROGRAM STUDI INFORMATIKA

UNIVERSITAS MULAWARMAN

SAMARINDA

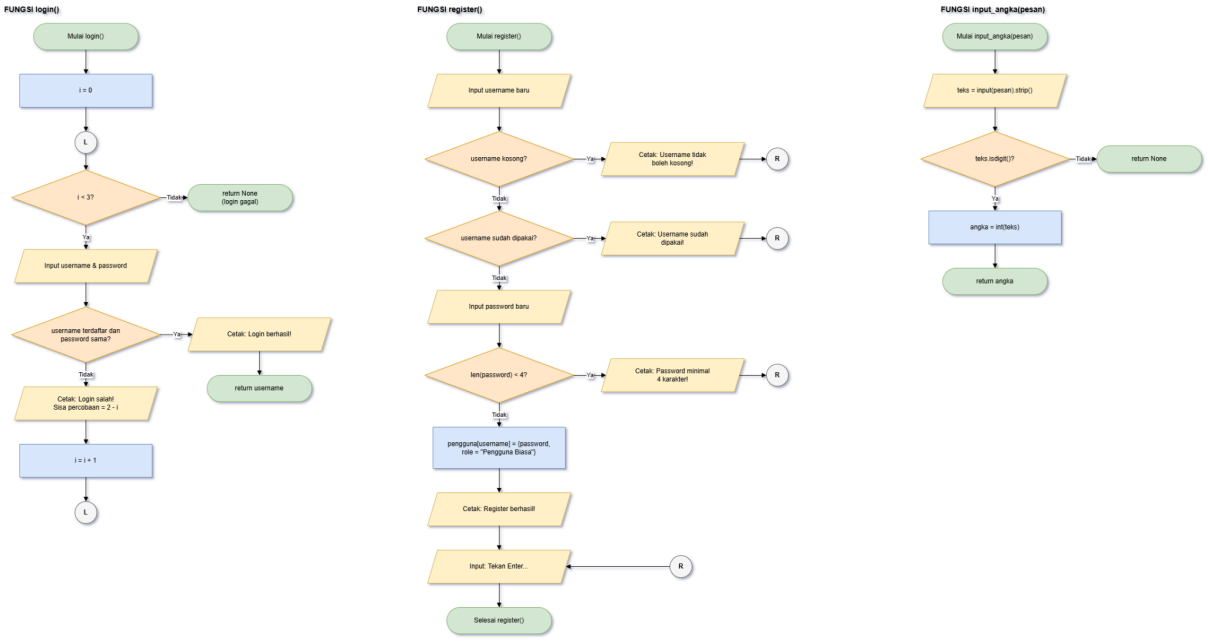
2026

1. Flowchart



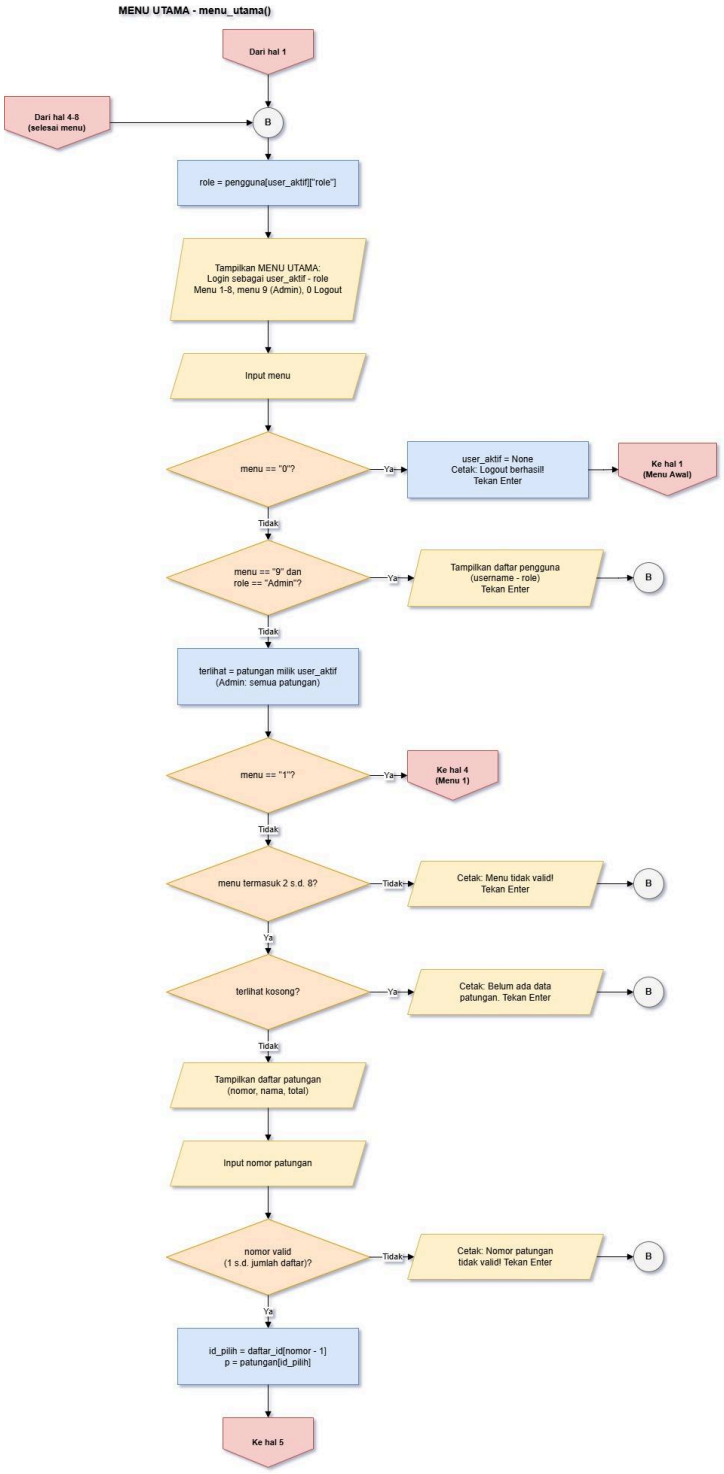
Gambar 1.1 *Flowchart Alur Utama Program Kalkulator Patungan*

Gambar 1.1 menunjukkan alur utama program yang dimulai dari inisialisasi data, kemudian pengguna memilih menu login, register, atau keluar.



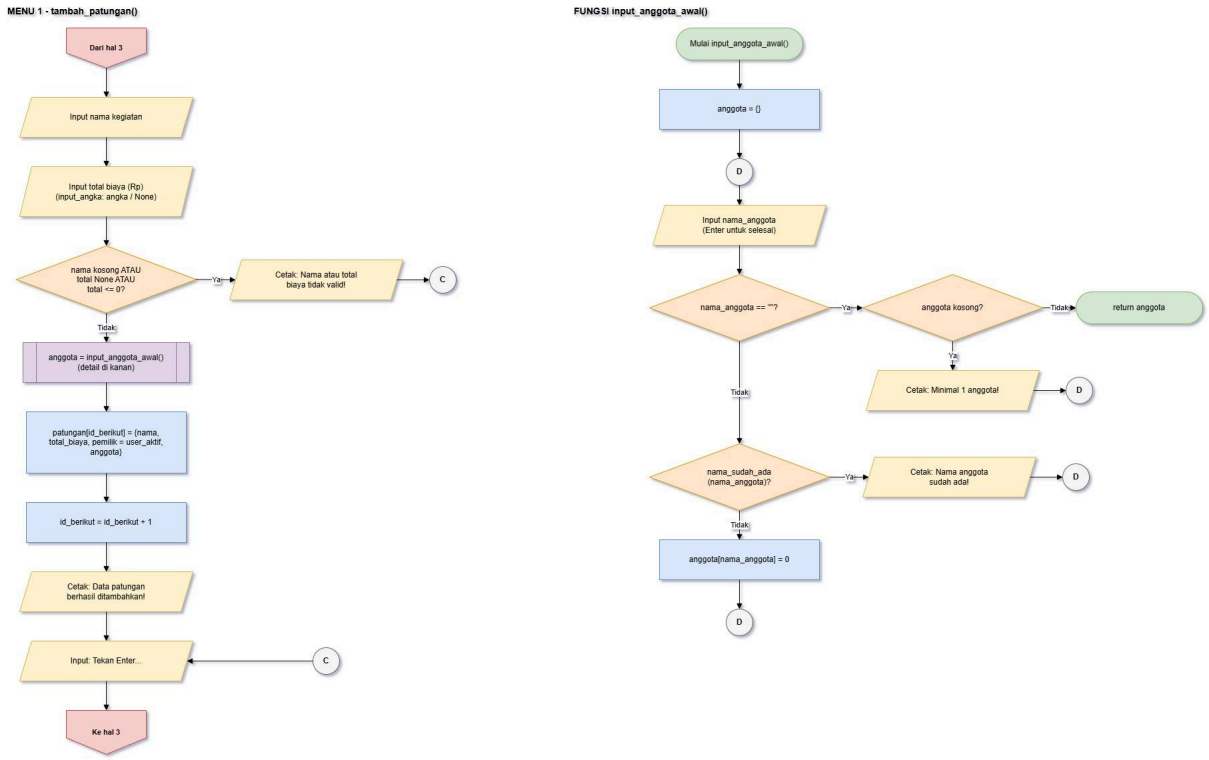
Gambar 1.2 *Flowchart Login, Register, dan Input Angka*

Gambar 1.2 menjelaskan proses login, pembuatan akun baru, serta validasi input angka agar sesuai dengan ketentuan program.



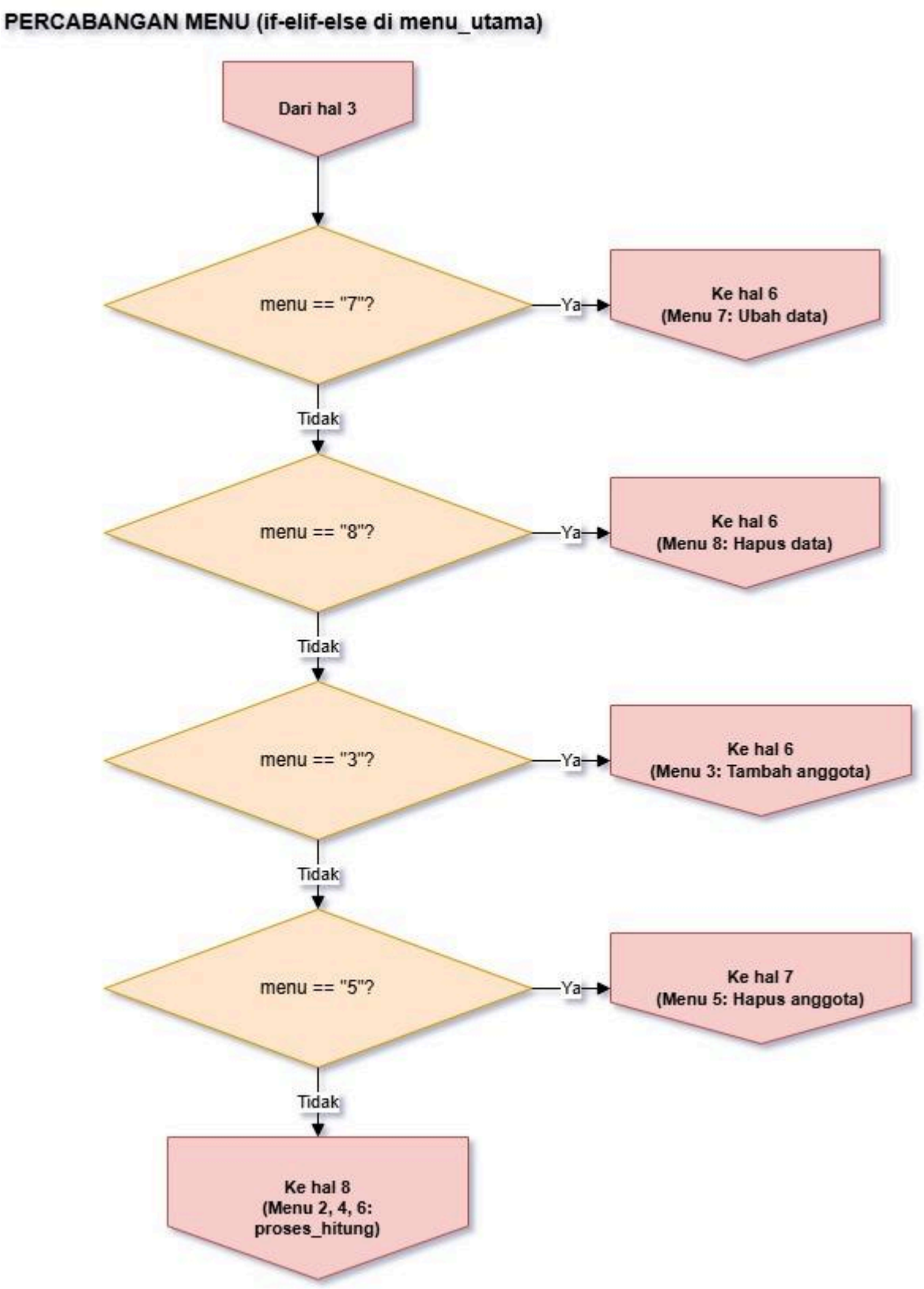
Gambar 1.3 *Flowchart Menu Utama*

Gambar 1.3 menunjukkan menu utama yang dapat diakses pengguna setelah berhasil login, sesuai dengan peran masing-masing.



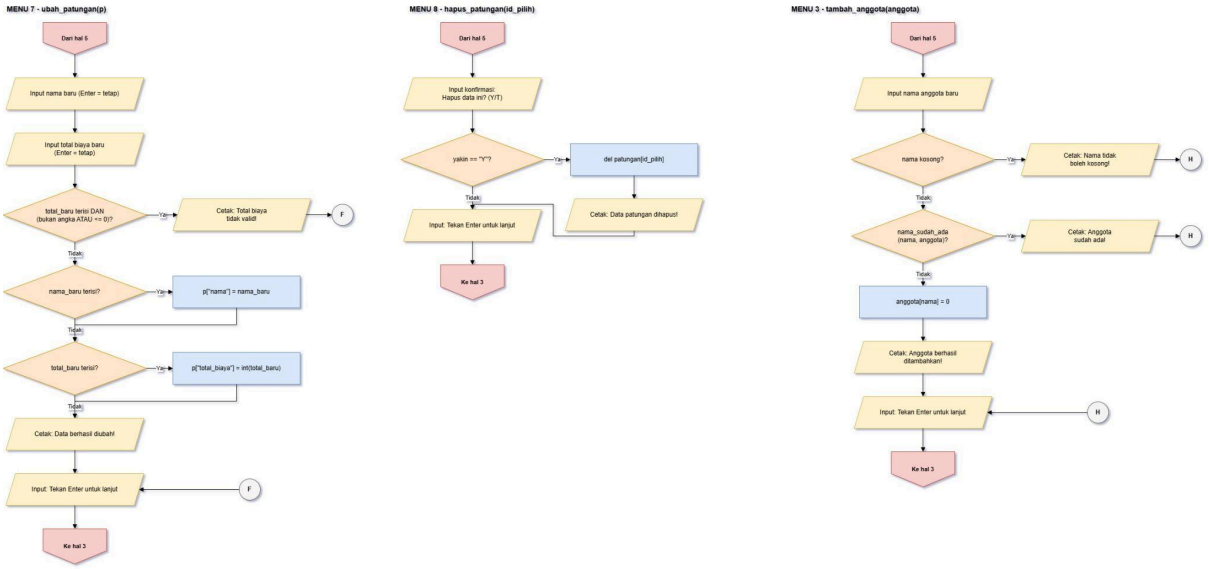
Gambar 1.4 *Flowchart Menu Tambah Patungan*

Gambar 1.4 menjelaskan proses menambahkan data patungan dengan memasukkan nama kegiatan, total biaya, dan anggota.



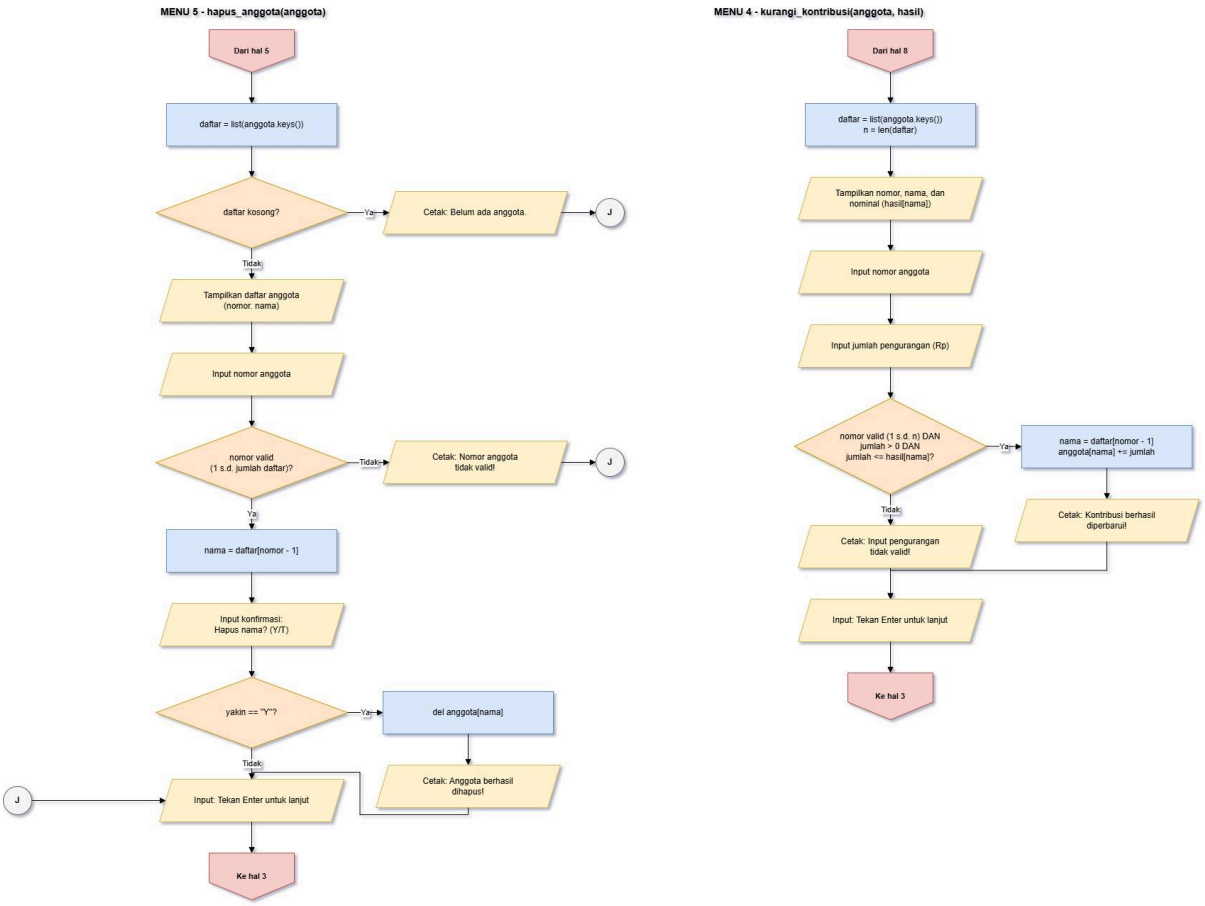
Gambar 1.5 *Flowchart Pemilihan Menu*

Gambar 1.5 menjelaskan proses pemilihan fitur untuk mengelola data patungan dan menjalankan perhitungan.



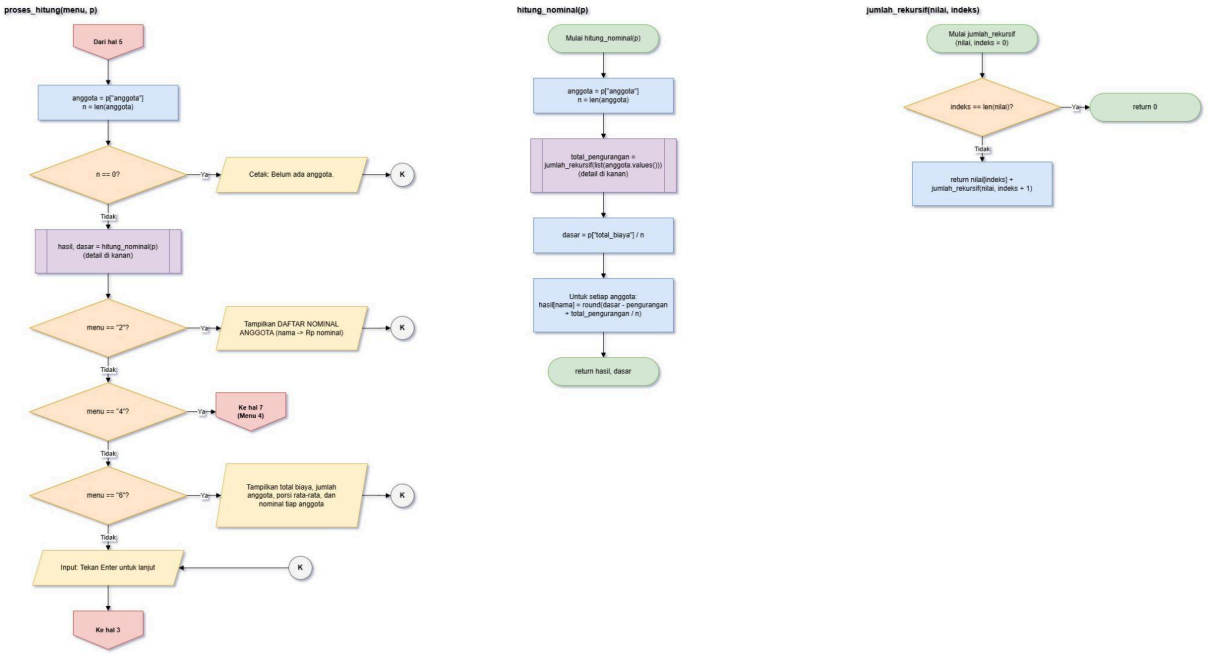
Gambar 1.6 Flowchart Ubah, Hapus, dan Tambah Anggota

Gambar 1.6 menjelaskan proses mengubah data patungan, menghapus data, dan menambahkan anggota baru.



Gambar 1.7 Flowchart Hapus Anggota dan Pengurangan Kontribusi

Gambar 1.7 menjelaskan proses menghapus anggota serta mengurangi nominal kontribusi anggota sesuai input pengguna.



Gambar 1.8 Flowchart Perhitungan Nominal Patungan

Gambar 1.8 menjelaskan proses menghitung nominal patungan setiap anggota dan menampilkan hasil perhitungan.

2. Deskripsi Singkat Program

Program ini adalah aplikasi konsol Python bernama Sistem Kalkulator Patungan yang digunakan untuk mengelola pembagian biaya bersama dalam suatu kegiatan. Program menyediakan menu Login, Register, dan Keluar. Login diberikan maksimal tiga kali percobaan. Pengguna juga dapat membuat akun baru dengan username yang belum digunakan dan password minimal empat karakter.

Setelah login, pengguna dapat menambahkan data patungan, menampilkan daftar anggota, menambahkan anggota, mengurangi kontribusi anggota, menghapus anggota, menghitung nominal patungan, mengubah data patungan, dan menghapus data patungan. Program memiliki dua jenis hak akses, yaitu Admin dan Pengguna Biasa. Admin dapat melihat daftar seluruh pengguna dan mengakses semua data patungan, sedangkan Pengguna Biasa hanya dapat mengakses data patungan miliknya sendiri.

Data program disimpan menggunakan dictionary, yaitu dictionary pengguna untuk menyimpan akun, dictionary patungan untuk menyimpan data kegiatan patungan, serta dictionary anggota untuk menyimpan nama anggota dan jumlah kontribusinya. Program juga

menggunakan fungsi rekursif jumlah_rekursif() untuk menghitung total kontribusi anggota dan hitung_nominal() untuk menghitung nominal patungan setiap anggota.

Program dilengkapi dengan validasi input menggunakan try-except, pemeriksaan nama anggota agar tidak duplikat, serta konfirmasi sebelum menghapus data. Seluruh data hanya disimpan di memori sehingga akan hilang ketika program ditutup.

3. Source Code

A. Variabel Global dan Data Awal

Bagian ini berisi import os untuk membersihkan layar, dictionary pengguna untuk menyimpan akun, dictionary patungan untuk menyimpan data kegiatan, serta variabel id_berikut sebagai penghitung ID patungan dan user_aktif untuk menyimpan pengguna yang sedang login.

```
import os

bersih = "cls" if os.name == "nt" else "clear"

pengguna = {
    "admin": {"password": "admin123", "role": "Admin"},
    "zaky": {"password": "zaky", "role": "Pengguna Biasa"}
}

patungan = {}
id_berikut = 1
user_aktif = None
```

B.Fitur Login dan Register

Bagian ini berisi fungsi untuk memvalidasi input angka, menghitung total kontribusi menggunakan rekursi, memeriksa nama yang sudah ada, dan menghitung nominal patungan

setiap anggota. Fungsi input_angka() memastikan input berupa angka bulat. Fungsi jumlah_rekursif() menjumlahkan nilai secara rekursif, sedangkan hitung_nominal() menghitung nominal berdasarkan total biaya dan kontribusi anggota.

```
def login():
    for i in range(3):
        username = input("Username: ").strip()
        password = input("Password: ").strip()

        if (username in pengguna and
                pengguna[username]["password"] == password):
            print("Login berhasil!")
            return username

        print("Login gagal!")

    return None


def register():
    username = input("Username baru: ").strip()
    password = input("Password baru: ").strip()

    if not username or username in pengguna:
        print("Username tidak valid atau sudah digunakan!")
    elif len(password) < 4:
        print("Password minimal 4 karakter!")
    else:
        pengguna[username] = {
            "password": password,
            "role": "Pengguna Biasa"
        }
        print("Register berhasil!")

def register():
    username = input("Username baru: ").strip()
    password = input("Password baru: ").strip()

    if not username or username in pengguna:
        print("Username tidak valid atau sudah digunakan!")
    elif len(password) < 4:
        print("Password minimal 4 karakter!")
    else:
        pengguna[username] = {
            "password": password,
            "role": "Pengguna Biasa"
        }
```

```
        print("Register berhasil!")
```

C. Fitur Tambah Data Patungan

Fitur ini digunakan untuk memasukkan nama kegiatan, total biaya, dan nama anggota. Data yang sudah dimasukkan disimpan dalam dictionary patungan, sedangkan anggota disimpan dalam dictionary anggota. Program juga memeriksa agar nama anggota tidak duplikat.

```
def tambah_patungan():
    global id_berikut

    nama = input("Nama kegiatan: ").strip()

    try:
        total = int(input("Total biaya: Rp"))
        if not nama or total <= 0:
            raise ValueError
    except ValueError:
        print("Input tidak valid!")
        return

    anggota = {}

    while True:
        nama_anggota = input(
            "Nama anggota (Enter selesai): "
        ).strip()

        if not nama_anggota:
            if anggota:
                break
            print("Minimal satu anggota!")
            continue

        if nama_anggota.lower() in [
            n.lower() for n in anggota
        ]:
            print("Anggota sudah ada!")
        else:
            anggota[nama_anggota] = 0

    patungan[id_berikut] = {
        "nama": nama,
        "total": total,
```

```
        "pemilik": user_aktif,
        "anggota": anggota
    }

    id_berikut += 1
    print("Patungan berhasil ditambahkan!")
```

D. Fitur Menghitung Nominal Patungan

Fitur ini menghitung pembagian biaya berdasarkan total biaya dan jumlah anggota. Fungsi hitung_patungan() menampilkan nama kegiatan, total biaya, jumlah anggota, serta nominal yang harus dibayar setiap anggota.

```
def hitung_patungan(p):
    anggota = p["anggota"]
    jumlah = len(anggota)

    if jumlah == 0:
        print("Belum ada anggota!")
        return

    nominal = p["total"] // jumlah

    print("Nama kegiatan:", p["nama"])
    print("Total biaya: Rp", p["total"])
    print("Jumlah anggota:", jumlah)

    for nama in anggota:
        print(nama, "membayar Rp", nominal)
```

E. Menu Utama dan Program Utama

Bagian ini mengatur menu Login, Register, dan Keluar. Setelah login berhasil, pengguna dapat memilih Tambah Patungan, Hitung Patungan, atau Logout. Program juga menggunakan try-except untuk menangani kesalahan input dan finally untuk menampilkan pesan ketika program selesai.

```
def main():
    global user_aktif

    while True:
        os.system(bersih)
        print("=== KALKULATOR PATUNGAN ===")
```

```
        print("1. Login")
        print("2. Register")
        print("3. Keluar")

        pilih = input("Pilih menu: ")

        if pilih == "1":
            user_aktif = login()

            if user_aktif is None:
                input("Login gagal. Enter...")
                continue

            while True:
                print("\n=== MENU UTAMA ===")
                print("1. Tambah Patungan")
                print("2. Hitung Patungan")
                print("0. Logout")

                menu = input("Pilih menu: ")

                if menu == "0":
                    user_aktif = None
                    break

                elif menu == "1":
                    tambah_patungan()

                elif menu == "2":
                    daftar = {
                        k: v for k, v in patungan.items()
                        if v["pemilik"] == user_aktif
                    }

                    if not daftar:
                        print("Belum ada patungan!")
                    else:
                        for nomor, p in daftar.items():
                            print(nomor, "-", p["nama"])

                        try:
                            nomor = int(
                                input("Pilih ID patungan: ")
                            )

                            if nomor in daftar:
                                hitung_patungan(daftar[nomor])
                            else:
```

```
                                print("ID tidak ditemukan!")

                        except ValueError:
                            print("Masukkan angka!")

                else:
                    print("Menu tidak valid!")

                input("Tekan Enter...")

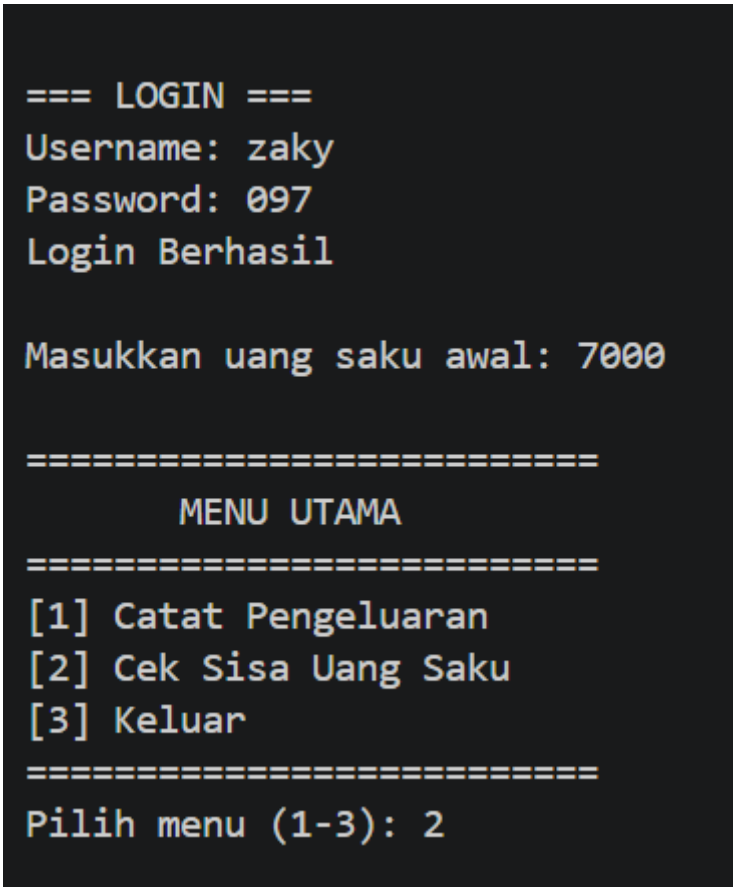
        elif pilih == "2":
            register()
            input("Tekan Enter...")

        elif pilih == "3":
            print("Terima kasih!")
            break

        else:
            print("Menu tidak valid!")
            input("Tekan Enter...")


try:
    main()
except (KeyboardInterrupt, EOFError):
    print("\nProgram dihentikan.")
finally:
    print("Program selesai.")
```

4. Hasil Output



Gambar 4.1 output

5. Git Add

git add adalah perintah Git yang digunakan untuk menambahkan perubahan file ke dalam staging area. Staging area merupakan tempat untuk menyiapkan perubahan yang akan disimpan ke dalam riwayat Git menggunakan perintah git commit.

```
PS C:\praktikum-apd-2026\praktikum-apd-26> git add .
```

Gambar 5.1 *Git Add*

6. Git Commit

git commit adalah perintah Git yang digunakan untuk menyimpan perubahan yang sudah berada di staging area ke dalam riwayat repositori lokal. Setiap commit memiliki pesan yang

menjelaskan perubahan yang dilakukan sehingga pengguna dapat mengetahui perkembangan program dari waktu ke waktu.

```
PS C:\praktikum-apd-2026\praktikum-apd-26> git commit -m "done"
[main b5718bd] done
 7 files changed, 463 insertions(+), 1 deletion(-)
 create mode 100644 kelas/praktikum6.py/praktikum6.py
 create mode 100644 kelas/praktikum6.py/praktikum8.py
 create mode 100644 kelas/proyek/__pycache__/login.cpython-313.pyc
 create mode 100644 kelas/proyek/login.py
 create mode 100644 kelas/proyek/main.py
 create mode 100644 posttest/posttest-apd-7/2609106097-Muhammmadilhamzaky-PT-7.py
PS C:\praktikum-apd-2026\praktikum-apd-26>
```

Gambar 5.2 *Git commit*

7. Git Push

git push adalah perintah Git yang digunakan untuk mengirim commit dari repositori lokal ke repositori jarak jauh (remote repository), misalnya GitHub. Dengan perintah ini, perubahan yang sudah disimpan melalui git commit dapat diunggah sehingga tersedia di repositori online.

```
Successfully rebased and updated refs/heads/main.
PS C:\praktikum-apd-2026\praktikum-apd-26> git push
Enumerating objects: 20, done.
Counting objects: 100% (20/20), done.
Delta compression using up to 8 threads
Compressing objects: 100% (12/12), done.
Writing objects: 100% (15/15), 4.38 KiB | 897.00 KiB/s, done.
Total 15 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
To https://github.com/097zakyyy/praktikum-apd-26.git
   37fde9f..1ee8022  main -> main
PS C:\praktikum-apd-2026\praktikum-apd-26>
```

Gambar 5.3 *Git push*